

Collective Robotics – Tutorial 1

Scalability and Synchronization

Hochschule Bonn-Rhein-Sieg
Prof. Dr. Javad Ghofrani

Bhavesch Gandhi

Jaqueline Machida

Deadline: 17.05.2026

Contents

1	Task 1 – Scaling of a Computer System	2
1.1	Overview	2
1.2	Implementation	2
1.3	Subtask a – Poisson PMF	2
1.4	Subtask b – Sampling	3
1.5	Subtask c – Single Simulation Run	3
1.6	Subtask d – Average Queue Length vs. α (4 steps/job)	4
1.7	Subtask e – Comparison: 2 vs. 4 Steps per Job	5
2	Task 2 – Synchronization of a Swarm	6
2.1	Overview	6
2.2	Implementation	6
2.3	Subtask a – Flashing Over Time for Different Radii	6
2.4	Subtask b – Synchronization Amplitude vs. Vicinity Radius	7
3	Completed Tasks	9

1 Task 1 – Scaling of a Computer System

1.1 Overview

We model a computer job queue using a **Poisson arrival process**. New jobs arrive at rate α per time step; since we set $\Delta t = 1$, the parameter becomes $\lambda = \alpha$. The probability that exactly i jobs arrive in one time step is:

$$P(X = i) = \frac{e^{-\lambda} \lambda^i}{i!}, \quad \lambda = \alpha \Delta t = \alpha. \quad (1)$$

The server processes jobs one at a time, each taking a fixed number of steps (the *processing duration*). We are interested in how the average waiting queue length depends on α .

1.2 Implementation

All core logic lives in `task.1/queue_model.py` and is shared by all subtask scripts.

Poisson PMF (`poisson_pmf`). Evaluates Equation (1) analytically using Python’s `math` module.

Sampler (`sample_poisson`). Draws samples from the Poisson distribution using Knuth’s inversion algorithm: starting from a uniform random product $p \leftarrow \prod U_k$, we count how many factors are needed before p falls below $e^{-\lambda}$. This requires only the standard `random` module.

```

1 def sample_poisson(alpha):
2     lam = alpha
3     L = math.exp(-lam)
4     k, p = 0, 1.0
5     while p > L:
6         p *= random.random()
7         k += 1
8     return k - 1

```

Queue simulation (`simulate_queue`). Each of the N time steps proceeds in two phases:

1. **Arrival phase:** sample the number of new jobs and add them to the queue.
2. **Processing phase:** if the server is busy, decrement its remaining work counter; otherwise, if the queue is non-empty, dequeue one job and start processing it for `processing_duration` steps.

The function returns the time-average queue length.

1.3 Subtask a – Poisson PMF

Figure 1 shows $P(X = i)$ for $\alpha \in \{0.01, 0.1, 0.5, 1\}$. For very small α the mass is concentrated near $i = 0$ (almost no jobs arrive per step). As α increases the distribution spreads

and its mode shifts to the right, as expected from a Poisson distribution whose mean and variance both equal $\lambda = \alpha$.

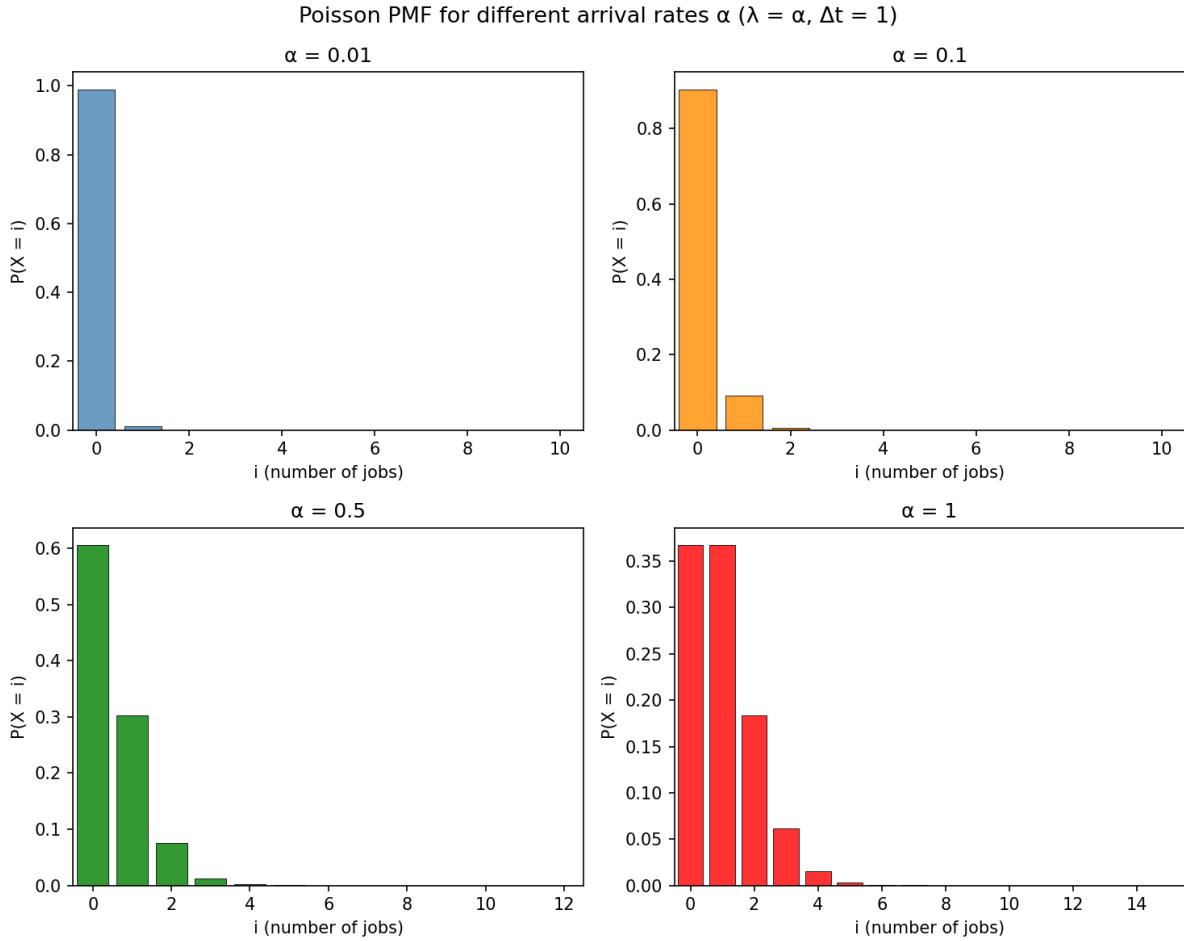


Figure 1: Poisson PMF $P(X = i)$ for four arrival rates α .

1.4 Subtask b – Sampling

Figure 2 compares 10 000 samples drawn with `sample_poisson` ($\alpha = 0.5$) against the analytical PMF. The empirical bars closely match the theoretical curve, confirming that the sampler is correct.

1.5 Subtask c – Single Simulation Run

We ran the queue model for 2000 time steps with $\alpha = 0.1$ and a processing duration of 4 steps per job. The resulting **average queue length was approximately 0.16 jobs**.

This low value is expected: the average arrival rate is $\alpha = 0.1$ jobs/step, while the server can process $1/4 = 0.25$ jobs/step, so the system is well below saturation and jobs are rarely queued.

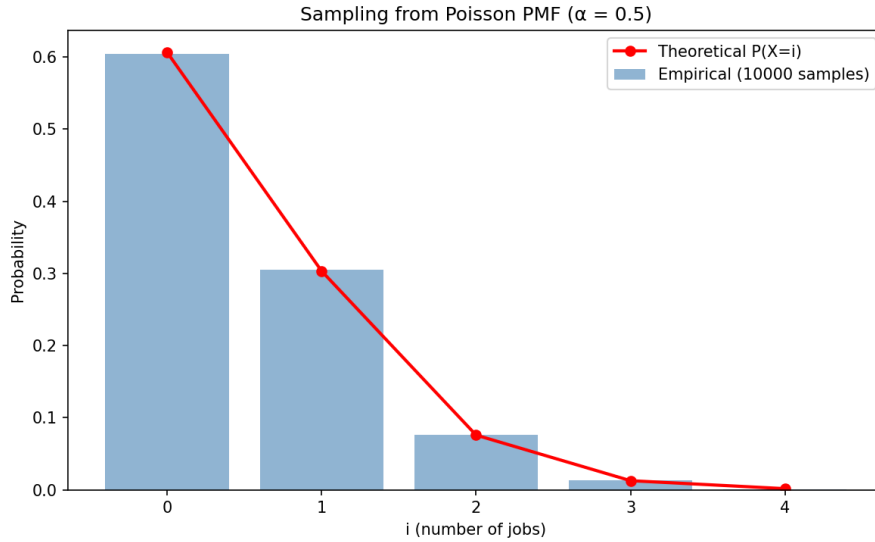


Figure 2: Empirical frequency vs. theoretical PMF for $\alpha = 0.5$, 10 000 samples.

1.6 Subtask d – Average Queue Length vs. α (4 steps/job)

We swept $\alpha \in [0.005, 0.25]$ in steps of 0.005. For each value we ran 200 independent 2000-step simulations and averaged the resulting queue lengths to obtain one representative point per α (see Figure 3).

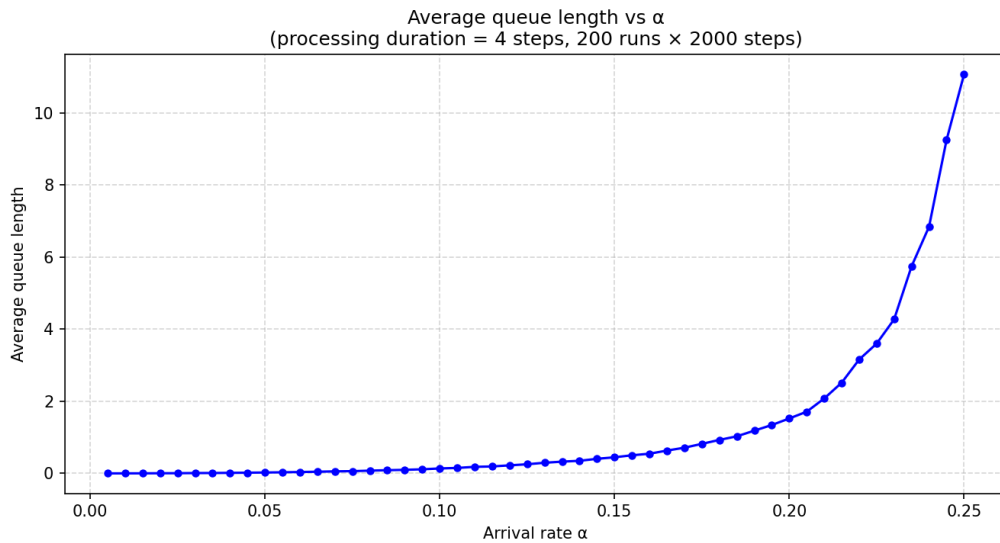


Figure 3: Average queue length as a function of α for a processing duration of 4 steps per job (200 runs $\times$ 2000 steps each).

Discussion. The curve is nearly flat and close to zero for small α , then rises steeply as α approaches 0.25. This is a classic queuing-theory result: the system is stable when the arrival rate is below the service rate ($\alpha < 1/\text{processing-duration} = 0.25$), and the queue diverges as $\alpha \rightarrow 0.25$ from below. The sharp increase near saturation is characteristic of an M/D/1 queue (Poisson arrivals, deterministic service, one server).

1.7 Subtask e – Comparison: 2 vs. 4 Steps per Job

Figure 4 shows the same experiment for a processing duration of **2 steps/job** with $\alpha \in [0.005, 0.5]$ alongside the 4-step case.

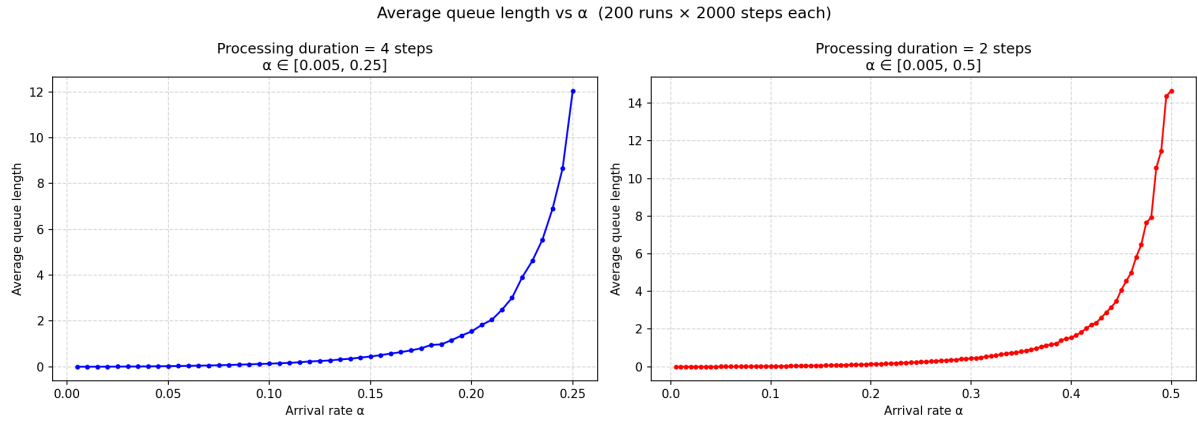


Figure 4: Side-by-side comparison of average queue length vs. α for processing durations of 4 (left) and 2 (right) steps per job.

Discussion. Halving the processing duration doubles the server throughput from 0.25 to 0.50 jobs/step. Consequently the critical arrival rate at which the queue explodes shifts from ≈ 0.25 to ≈ 0.50 . The shape of the curve is identical – a flat region followed by a steep rise near saturation – but the entire curve is stretched along the α axis by a factor of two. This confirms that the saturation threshold is governed purely by the ratio $\alpha/(1/\text{processing_duration})$.

2 Task 2 – Synchronization of a Swarm

2.1 Overview

We simulate a population of $N = 150$ fireflies scattered uniformly in a 1×1 square. Each firefly has a clock of length $L = 50$ steps: it flashes for the first $L/2 = 25$ steps and is dark for the remaining 25. Fireflies interact locally: one step after starting to flash (i.e., at `clock = 1`), a firefly checks its neighbours within radius r and applies a **local majority rule** – if more than half of its neighbours are currently flashing, it advances its own clock by an extra step, which causes it to flash one step earlier in the next cycle.

2.2 Implementation

Firefly class (`firefly.py`). Each firefly stores its (x, y) position (drawn from $\mathcal{U}[0, 1]^2$ at creation), its current clock value, and the cycle length L . Three helper methods implement the task logic:

- `is_flashing()` – returns `True` when `clock < L/2`.
- `should_check_neighbors()` – returns `True` when `clock == 1` (one step after starting to flash).
- `corrects_clock()` – increments clock by 1 (correction step).
- `tick()` – advances clock by 1 and wraps modulo L .

Simulation loop (`simulate_2a.py`, `simulate_2b.py`). Each time step proceeds in three phases:

1. **Observation:** fireflies at `clock == 1` count their flashing neighbours within radius r ; if a strict majority is flashing they are marked for correction.
2. **Correction:** all marked fireflies call `corrects_clock()`.
3. **Tick:** all fireflies call `tick()`.

2.3 Subtask a – Flashing Over Time for Different Radii

Figure 5 shows the number of concurrently flashing fireflies over 5000 time steps for $r \in \{0.05, 0.1, 0.5, 1.4\}$.

Discussion.

- $r = 0.05$ (**avg. 0.92 neighbours**): Almost no connectivity. The count oscillates randomly around $N/2 = 75$ with no visible synchronization.
- $r = 0.1$ (**avg. 4.04 neighbours**): Still too sparse; weak local clustering but no global synchronization.
- $r = 0.5$ (**avg. 72 neighbours**): Clear synchronization emerges around $t \approx 1000$. The population transitions to coherent oscillations between near-0 and near-150.
- $r = 1.4$ (**avg. 149 neighbours**): Nearly fully connected. Synchronization is fastest and most stable, with maximum amplitude.

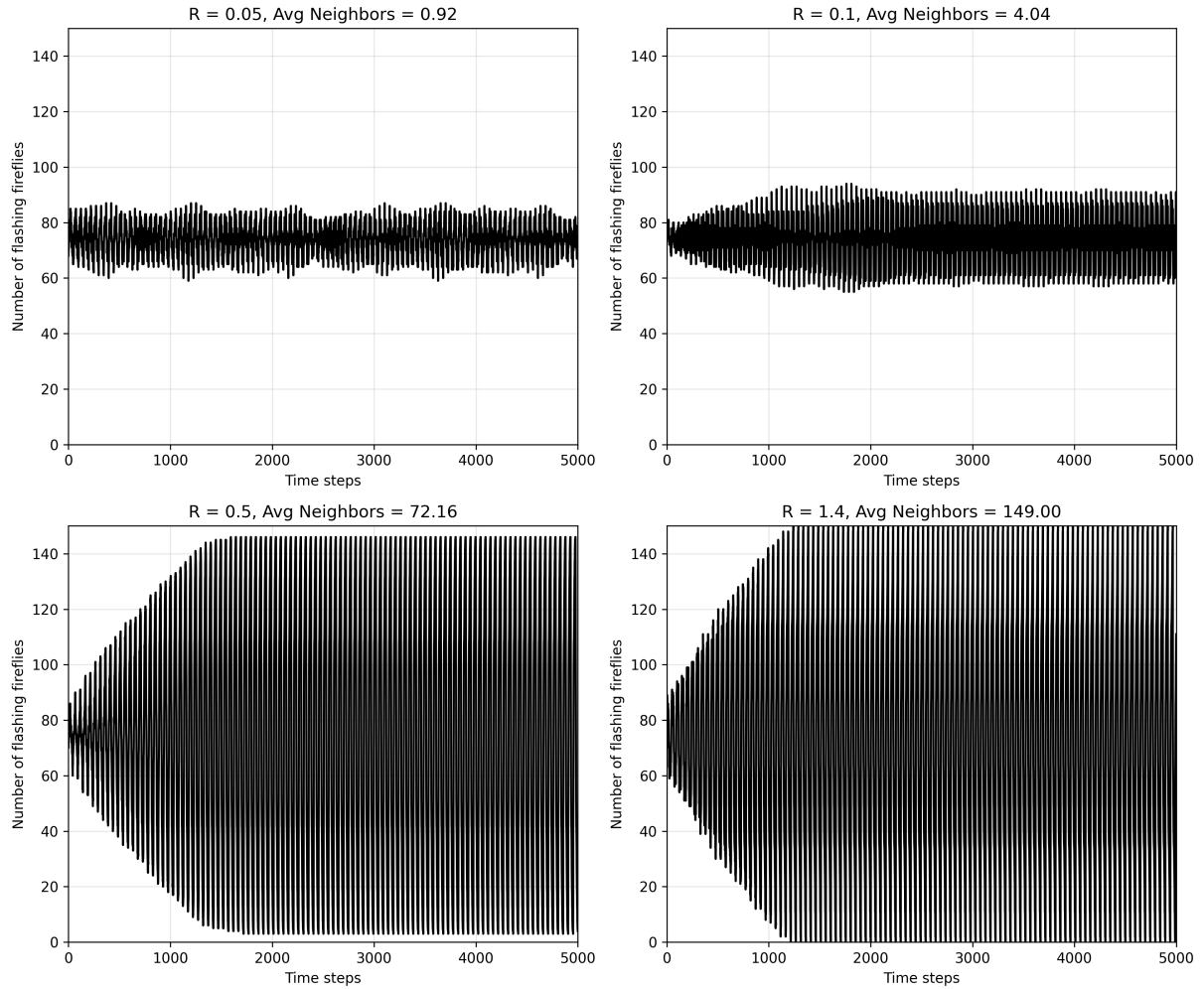


Figure 5: Number of flashing fireflies over time for four vicinity radii. $N = 150$, $L = 50$, 5000 time steps.

A critical connectivity threshold lies between $r = 0.1$ and $r = 0.5$; below it local interactions are insufficient to propagate phase corrections across the swarm.

2.4 Subtask b – Synchronization Amplitude vs. Vicinity Radius

The amplitude of the last cycle (last $L = 50$ steps) is defined as $\max(\text{flashing}) - \min(\text{flashing})$. We averaged this measure over 50 independent simulation runs for each $r \in [0.025, 1.4]$ (step 0.025). The result is shown in Figure 6.

Discussion. Three regions are visible:

1. $r < 0.3$: Amplitude rises steeply from ≈ 20 to ≈ 100 . Connectivity is insufficient for global sync; only small local clusters form.
2. $r \approx 0.3\text{--}0.5$: Amplitude plateaus at its maximum ($\approx 140\text{--}148$). This is the optimal operating range.
3. $r > 0.5$: Amplitude remains high. Additional connectivity does not improve synchronization – diminishing returns.

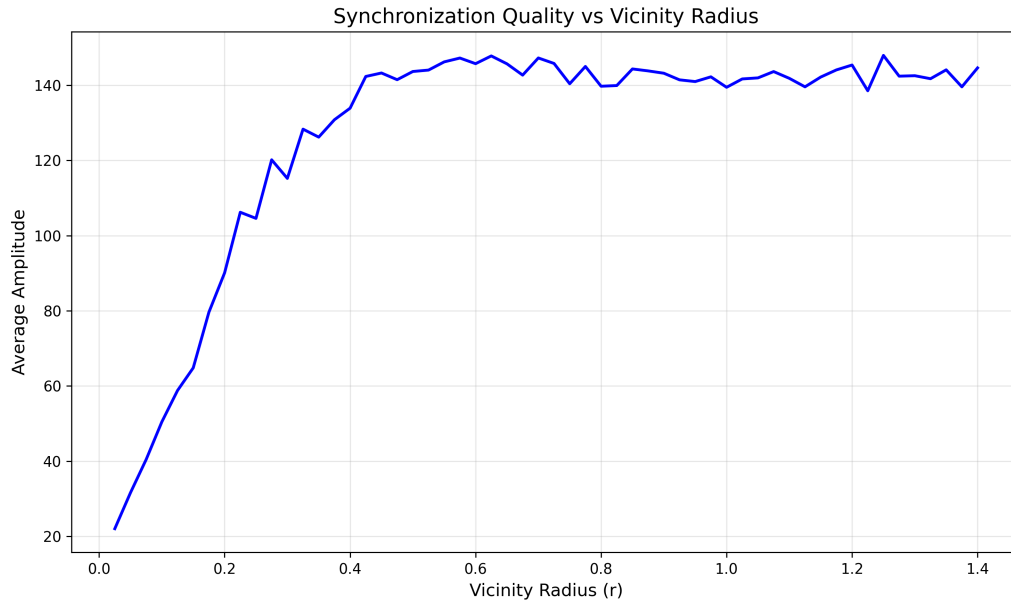


Figure 6: Average synchronization amplitude vs. vicinity radius r (50 runs $\times$ 5000 steps each).

Good choice for vicinity and swarm density. For a swarm density of 150 fireflies in a 1×1 square, a vicinity radius of $r \approx 0.3$ – 0.4 is optimal. It provides enough neighbours (~ 20 – 40) for reliable majority voting while keeping the sensing range small (energy-efficient, bio-plausible).

Interpretation of amplitude. A *low amplitude* means fireflies flash nearly independently: at any moment, roughly half the swarm flashes (~ 75 out of 150), with no coherent peaks. A *high amplitude* means the swarm flashes collectively: almost all fireflies flash simultaneously, and the count swings close to 0 and 150 each cycle. Amplitude is therefore a direct measure of *synchronization quality*.

3 Completed Tasks

Task	Description	Status
1a	Plot Poisson PMF for $\alpha \in \{0.01, 0.1, 0.5, 1\}$	✓
1b	Sampler from $P(X = i)$, empirical vs. theoretical plot	✓
1c	2000-step simulation, $\alpha = 0.1$, 4 steps/job	✓
1d	Avg. queue length vs. $\alpha \in [0.005, 0.25]$, 200 runs	✓
1e	Comparison: 2-step vs. 4-step processing	✓
2a	Flashing over time for $r \in \{0.05, 0.1, 0.5, 1.4\}$	✓
2b	Amplitude vs. $r \in [0.025, 1.4]$, 50 runs	✓